

Subarray Sum = K

1. Overview

Subarray Sum = K is a classic prefix-sum + hashing pattern used to count or detect contiguous subarrays whose sum is exactly k .

A subarray is continuous. For example, in $[1, 2, 3, 4]$, $[2, 3]$ is a subarray but $[1, 3]$ is not.

The main idea is:

- Convert every subarray sum query into a difference of prefix sums.
- Use a hash map to remember prefix sums seen so far.
- Find how many previous prefix sums would make the current subarray sum equal to k .

This is heavily used in placements, online assessments, and competitive programming because it reduces an $O(n^2)$ brute force approach to $O(n)$.

2. Intuition

Let $\text{prefix}[i]$ mean the sum of elements from index 0 to i .

The sum of subarray $l..r$ is:

$$\text{prefix}[r] - \text{prefix}[l - 1]$$

We want:

$$\text{prefix}[r] - \text{prefix}[l - 1] = k$$

Rearrange:

$$\text{prefix}[l - 1] = \text{prefix}[r] - k$$

So while scanning the array, if the current prefix sum is currentSum , we need to know how many previous prefix sums equal $\text{currentSum} - k$.

Analogy: imagine walking on a number line. Your current position is the prefix sum. You want to know whether you were previously at a position exactly k units behind your current position.

Why it works:

1. Every subarray ending at current index has some starting point before or at current index.
2. The sum of that subarray is the difference between two prefix sums.

3. A hash map lets us count valid starting points in constant average time.

3. When to Use It

Use this pattern when:

- The problem asks for subarray sum equal to k .
- The array may contain negative numbers.
- You need to count number of valid subarrays.
- You need to check if at least one valid subarray exists.
- You need longest/shortest subarray with exact sum and negative values are allowed.

Common trigger phrases:

- "Number of subarrays with sum K "
- "Continuous subarray"
- "Contiguous segment"
- "Sum equals target"
- "Subarray divisible by K "
- "Binary subarray with sum"
- "Count subarrays with given XOR" with XOR variation

4. When Not to Use It

Do not use this blindly when:

- The problem asks for subsequences, not subarrays.
- The array has only positive numbers and asks for shortest/longest sum condition; sliding window may be simpler.
- The condition is $\text{sum} \leq k$ or $\text{sum} \geq k$ with negative values; plain prefix hash does not directly solve inequality.
- You need range updates and many queries; use Fenwick Tree or Segment Tree.
- The input is tiny and brute force is acceptable, though interviews usually expect optimization.

Common wrong assumptions:

- Sliding window works with negative numbers. It usually does not for exact sum.
- Storing only one prefix sum occurrence is enough for counting. You need frequencies.
- Forgetting prefix sum 0 before the scan misses subarrays starting at index 0 .

5. Core Concepts

Prefix Sum

Prefix sum stores cumulative sum up to current index.

Example:

```
arr = [1, 2, 3]
```

```
prefix = [1, 3, 6]
```

It matters because any subarray sum can be represented as a difference of two prefix sums.

Hash Map Frequency

The map stores how many times each prefix sum has appeared.

If $\text{currentSum} - k$ appeared x times, then x subarrays ending at current index have sum k .

Initial Prefix Sum Zero

Before reading elements, sum is 0 .

This handles cases where a subarray from index 0 itself has sum k .

Example:

```
arr = [3, 1], k = 3  
currentSum at index 0 = 3  
currentSum - k = 0
```

So $\text{prefixCount}[0]$ must already be 1 .

Negative Numbers

Negative numbers make window sums non-monotonic. Prefix sum hashing still works because it does not depend on monotonic movement.

6. Step-by-Step Algorithm

1. Create a hash map prefixCount .
2. Set $\text{prefixCount}[0] = 1$.
3. Set $\text{currentSum} = 0$ and $\text{answer} = 0$.
4. Traverse every element num .
5. Add num to currentSum .
6. Compute $\text{needed} = \text{currentSum} - k$.
7. Add $\text{prefixCount}[\text{needed}]$ to answer .
8. Increment $\text{prefixCount}[\text{currentSum}]$.
9. Return answer .

7. Dry Run

Input:

```
arr = [1, 2, 3, -2, 5]  
k = 3
```

Initial state:

```
currentSum = 0
prefixCount = {0: 1}
answer = 0
```

Index	num	currentSum	needed = currentSum - k	prefixCount[needed]	answer	Map update
0	1	1	-2	0	0	count[1]++
1	2	3	0	1	1	count[3]++
2	3	6	3	1	2	count[6]++
3	-2	4	1	1	3	count[4]++
4	5	9	6	1	4	count[9]++

Valid subarrays:

- [1, 2]
- [3]
- [2, 3, -2]
- [-2, 5]

Final answer: 4

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long countSubarraysWithSumK(const vector<int>& nums, long long k) {
    unordered_map<long long, long long> prefixCount;
    prefixCount[0] = 1;

    long long currentSum = 0;
    long long totalSubarrays = 0;

    for (int num : nums) {
        currentSum += num;

        long long neededPrefix = currentSum - k;
        if (prefixCount.count(neededPrefix)) {
            totalSubarrays += prefixCount[neededPrefix];
        }

        prefixCount[currentSum]++;
    }
}
```

```
        return totalSubarrays;
    }

    int main() {
        int n;
        long long k;
        cin >> n >> k;

        vector<int> nums(n);
        for (int i = 0; i < n; i++) {
            cin >> nums[i];
        }

        cout << countSubarraysWithSumK(nums, k) << '\n';
        return 0;
    }
```

Example input:

```
5 3
1 2 3 -2 5
```

Output:

```
4
```

pYTHON IMPLEMENTATION

```
from collections import defaultdict

def count_subarrays_with_sum_k(nums, k):
    prefix_count = defaultdict(int)
    prefix_count[0] = 1

    current_sum = 0
    total_subarrays = 0

    for num in nums:
        current_sum += num
        needed_prefix = current_sum - k
        total_subarrays += prefix_count[needed_prefix]
        prefix_count[current_sum] += 1

    return total_subarrays
```

```
n, k = map(int, input().split())
nums = list(map(int, input().split()))
print(count_subarrays_with_sum_k(nums, k))
```

10. Code Explanation

- `prefixCount[0] = 1` represents the empty prefix before the array starts.
- `currentSum += num` updates the prefix sum at the current index.
- `neededPrefix = currentSum - k` identifies the prefix sum that must have appeared earlier.
- `totalSubarrays += prefixCount[neededPrefix]` counts all subarrays ending at the current index with sum `k`.
- `prefixCount[currentSum]++` records the current prefix sum for future indices.

The code uses `long long` because sums and counts may exceed `int` range.

11. Complexity Analysis

Operation	Complexity
Preprocessing	$O(1)$
Single pass	$O(n)$
Hash lookup/update	$O(1)$ average
Overall time	$O(n)$ average
Worst-case hash time	$O(n^2)$ theoretically, rare with standard constraints
Space	$O(n)$

12. Common Patterns

Pattern	How to identify it	General approach	Example problems
Count exact sum	"count subarrays sum equals k "	Prefix sum frequency map	LeetCode 560
Existence of exact sum	"does any subarray sum to k "	Store seen prefix sums	GFG Subarray with Given Sum with negatives
Longest exact sum	"maximum length subarray sum k "	Store first index of prefix sum	LeetCode 325
Divisible sum	"sum divisible by k "	Store prefix modulo frequencies	LeetCode 974
Binary exact sum	binary array and target sum	Prefix count or at-most trick	LeetCode 930

13. Common Mistakes

- Forgetting `prefixCount[0] = 1`.
- Updating `prefixCount[currentSum]` before checking `currentSum - k`.
- Using `int` for large sums.
- Applying sliding window when negative numbers exist.
- Confusing subarray with subsequence.
- For longest length, storing latest index instead of first index.
- For modulo problems, not normalizing negative modulo.

14. Edge Cases

- Empty array: answer is `0`.
- Single element equal to `k`.
- Single element not equal to `k`.
- All zeros with `k = 0`.
- Negative values mixed with positives.
- Very large values causing overflow.
- Duplicate prefix sums.
- `k = 0`.

15. Variations

Variation	What changes	Placement/CP importance
Longest subarray sum K	Store first index of each prefix sum	Very important
Subarray sum divisible by K	Use prefix sum modulo K	Very important
Count subarrays with XOR K	Prefix XOR instead of sum	Important
Binary subarrays with sum K	Prefix count or sliding-window at-most	Important
2D submatrix sum K	Fix row pairs and apply 1D prefix hash	Advanced CP/interviews

16. Related Algorithms/Data Structures

- Sliding Window: better when all numbers are non-negative and condition is monotonic.
- Prefix Sum Array: good for static range sum queries.
- Fenwick Tree: useful for dynamic prefix sums or counting ordered prefix conditions.
- Segment Tree: useful for range updates or complex range queries.
- Hash Map: core structure for storing prefix frequencies.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Subarray with Given Sum	GFG	Prefix sum or sliding window depending on constraints	Easy
Find Pivot Index	LeetCode	Prefix/suffix sum relation	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Subarray Sum Equals K	LeetCode 560	Prefix sum frequency	Medium
Continuous Subarray Sum	LeetCode 523	Prefix modulo	Medium
Subarray Sums Divisible by K	LeetCode 974	Modulo frequency	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Number of Submatrices That Sum to Target	LeetCode 1074	Compress rows + prefix hash	Hard
Count of Range Sum	LeetCode 327	Prefix sums + merge sort/Fenwick	Hard
CSES Subarray Sum Queries variants	CSES	Prefix/range reasoning	Hard

18. Interview Explanation

"I use prefix sums because any subarray sum can be written as the difference between two prefix sums. While scanning the array, if my current prefix sum is S , I need a previous prefix sum $S - k$. I store frequencies of prefix sums in a hash map, so each index can be processed in constant average time. This also works with negative numbers, unlike a normal sliding window."

19. Revision Notes

- Key idea: `subarraySum(l..r) = prefix[r] - prefix[l - 1]`.
- Formula: need previous prefix `currentSum - k`.
- Initialize map with `{0: 1}`.
- Use `long long` in C++.
- Complexity: $O(n)$ time, $O(n)$ space.
- Trap: sliding window fails with negative numbers.

20. Final Cheat Sheet

Item	Reminder
When to use	Exact contiguous subarray sum, negatives allowed
Main operation	Prefix sum + frequency hash map
Complexity	$O(n)$ time, $O(n)$ space
Key code idea	<code>answer += count[currentSum - k]</code>
Edge cases	$k = 0$, all zeros, negatives, overflow

Longest Subarray With Condition

1. Overview

Longest subarray with condition is a family of problems where you need the maximum length of a contiguous segment satisfying some rule.

Examples:

- Longest subarray with sum $\leq k$
- Longest subarray with at most k distinct elements
- Longest subarray with equal number of 0s and 1s
- Longest subarray with sum exactly k
- Longest subarray where max-min $\leq \text{limit}$

The correct technique depends on the condition. Most placement problems fall into one of three patterns:

- Sliding window for monotonic conditions.
- Prefix sum + hash map for exact conditions with negative values.
- Deque or balanced structure for max/min constraints.

2. Intuition

Think of a subarray as a window with left and right boundaries.

For many conditions, we expand the right boundary to include more elements. If the condition becomes invalid, we move the left boundary until the window becomes valid again.

This works only when removing elements from the left can help restore validity in a predictable way.

Example:

```
Longest subarray with sum  $\leq k$   
arr contains only positive numbers
```

If sum becomes too large, moving left forward always decreases the sum. That makes sliding window valid.

But if negative numbers exist, removing from the left may increase or decrease the sum, so the simple window logic breaks.

Why it works:

1. Each element enters the window once.
2. Each element leaves the window once.
3. The window always represents the current best valid candidate ending at right .

3. When to Use It

Use this family of patterns when:

- The problem asks for maximum length subarray.
- The condition is about a contiguous segment.
- You can efficiently update the condition when adding/removing elements.
- The condition is monotonic under window shrinking.

Common trigger phrases:

- "Longest subarray"
- "Maximum length contiguous segment"
- "At most K"
- "No more than K distinct"
- "Sum less than or equal to K"
- "Longest substring without repeating characters"
- "Longest continuous subarray"

4. When Not to Use It

Avoid plain sliding window when:

- Negative values break monotonicity.
- You need exact sum with arbitrary integers.
- The condition depends on global ordering not maintainable by simple counts.
- The problem asks for subsequence, not subarray.
- You need minimum/maximum values and are not maintaining them efficiently.

Simpler alternatives:

- For exact sum with negatives, use prefix sum + hash map.
- For fixed length windows, use rolling sum.
- For offline range queries, use sorting/Fenwick/Segment Tree.

5. Core Concepts

Window Boundaries

`left` and `right` define the current subarray.

```
nums[left..right]
```

This matters because the answer is usually `right - left + 1`.

Validity Condition

The window must satisfy a condition such as:

- `sum <= k`
- `distinctCount <= k`
- `max - min <= limit`

Shrinking

When invalid, move `left` forward and update the window state.

Monotonicity

Sliding window requires that shrinking eventually fixes the invalid state.

For positive sums, shrinking decreases sum. For distinct count, shrinking decreases or preserves distinct count.

Data Structure for State

Different conditions need different state:

Condition	State
Sum	integer sum
Distinct elements	frequency map
Repeating characters	frequency map or last index
Max-min constraint	two monotonic deques

6. Step-by-Step Algorithm

For a common example, longest subarray with sum $\leq k$ and all elements non-negative:

1. Set `left = 0`, `currentSum = 0`, `bestLength = 0`.
2. Traverse `right` from `0` to `n - 1`.
3. Add `nums[right]` to `currentSum`.
4. While `currentSum > k`, subtract `nums[left]` and increment `left`.
5. Now the window is valid.
6. Update `bestLength = max(bestLength, right - left + 1)`.
7. Return `bestLength`.

7. Dry Run

Input:

```
nums = [2, 1, 3, 2, 1, 1]
k = 5
```

Goal: longest subarray with sum ≤ 5 .

right	nums[right]	currentSum after add	Shrink action	left	Valid window	bestLength
0	2	2	none	0	[2]	1
1	1	3	none	0	[2,1]	2

right	nums[right]	currentSum after add	Shrink action	left	Valid window	bestLength
2	3	6	remove 2	1	[1,3]	2
3	2	6	remove 1	2	[3,2]	2
4	1	6	remove 3	3	[2,1]	2
5	1	4	none	3	[2,1,1]	3

Final answer: 3.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int longestSubarraySumAtMostK(const vector<int>& nums, long long k) {
    int n = (int)nums.size();
    int left = 0;
    int bestLength = 0;
    long long currentSum = 0;

    for (int right = 0; right < n; right++) {
        currentSum += nums[right];

        while (left <= right && currentSum > k) {
            currentSum -= nums[left];
            left++;
        }

        bestLength = max(bestLength, right - left + 1);
    }

    return bestLength;
}

int main() {
    int n;
    long long k;
    cin >> n >> k;

    vector<int> nums(n);
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    cout << longestSubarraySumAtMostK(nums, k) << '\n';
    return 0;
}
```

Example input:

```
6 5
2 1 3 2 1 1
```

Output:

```
3
```

pYTHON IMPLEMENTATION

```
def longest_subarray_sum_at_most_k(nums, k):
    left = 0
    current_sum = 0
    best_length = 0

    for right, value in enumerate(nums):
        current_sum += value

        while left <= right and current_sum > k:
            current_sum -= nums[left]
            left += 1

        best_length = max(best_length, right - left + 1)

    return best_length

n, k = map(int, input().split())
nums = list(map(int, input().split()))
print(longest_subarray_sum_at_most_k(nums, k))
```

10. Code Explanation

- `left` marks the beginning of the current valid or repairable window.
- `right` expands the window one element at a time.
- `currentSum` stores the sum of the current window.
- The `while` loop shrinks the window until the sum is at most `k`.
- After shrinking, `right - left + 1` is a valid candidate length.

This implementation assumes all numbers are non-negative. If negative numbers are allowed, use a different method depending on the exact condition.

11. Complexity Analysis

Operation	Complexity
Expanding right pointer	$O(n)$ total
Moving left pointer	$O(n)$ total
Overall time	$O(n)$
Space	$O(1)$
Best case	$O(n)$
Worst case	$O(n)$

12. Common Patterns

Pattern	How to identify it	General approach	Example problems
At most K distinct	"at most k different values"	Sliding window + frequency map	LeetCode 340, 904
Sum at most K	non-negative values and sum constraint	Sliding window sum	GFG longest subarray sum $\leq K$
No repeats	"without repeating characters"	Window + frequency/last seen	LeetCode 3
Max-min $\leq$ limit	"absolute diff $\leq$ limit"	Two monotonic deques	LeetCode 1438
Exact sum with negatives	"sum exactly k" with negative values	Prefix sum + first index	LeetCode 325

13. Common Mistakes

- Using sliding window with negative numbers.
- Updating answer before restoring validity.
- Forgetting to decrement frequency when moving **left**.
- Not removing keys whose frequency becomes zero.
- Using **if** instead of **while** when multiple left moves are needed.
- Mixing up maximum length and minimum length update logic.
- Assuming every "longest subarray" problem is sliding window.

14. Edge Cases

- Empty input.
- Single valid element.
- Single invalid element.
- Whole array is valid.
- No valid non-empty subarray.
- $k = 0$.
- All equal values.
- Large values causing overflow.

- Negative values, if constraints allow them.

15. Variations

Variation	What changes	Placement/CP importance
Longest subarray with sum exactly K	Prefix sum + first index	Very important
Longest subarray with at most K distinct	Frequency map	Very important
Longest substring without repeats	Character frequency/last seen	Very important
Longest subarray max-min $\leq$ limit	Monotonic deque	Important
Longest balanced 0/1 subarray	Convert 0 to -1 and use prefix	Important

16. Related Algorithms/Data Structures

- Sliding Window: main technique for monotonic conditions.
- Prefix Sum: useful for exact sum and balance problems.
- Hash Map: tracks frequencies or first prefix positions.
- Monotonic Deque: maintains max/min in a moving window.
- Two Pointers: broader category that includes many window problems.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Maximum Average Subarray I	LeetCode 643	Fixed size window	Easy
Longest Harmonious Subsequence	LeetCode 594	Frequency reasoning	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Longest Substring Without Repeating Characters	LeetCode 3	Sliding window + set/map	Medium
Fruit Into Baskets	LeetCode 904	At most 2 distinct	Medium
Longest Repeating Character Replacement	LeetCode 424	Window with replacement budget	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Longest Continuous Subarray With Absolute Diff $\leq$ Limit	LeetCode 1438	Two deques	Hard

Problem	Platform	Main idea/pattern	Difficulty
Minimum Window Substring	LeetCode 76	Validity-count window	Hard
Count Subarrays With Fixed Bounds	LeetCode 2444	Boundary tracking	Hard

18. Interview Explanation

"For longest subarray problems, I first check whether the condition is monotonic. If expanding can make the window invalid and shrinking from the left can restore it, I use sliding window. I maintain the window state, move the right pointer to expand, move the left pointer while invalid, and update the maximum valid length after the window is fixed."

19. Revision Notes

- Key idea: maintain a valid window `[left, right]`.
- Update answer only after the condition is valid.
- Use `while`, not `if`, to repair invalid windows.
- Sliding window works best with monotonic conditions.
- Negative numbers often require prefix sum instead.
- Complexity: usually $O(n)$.

20. Final Cheat Sheet

Item	Reminder
When to use	Longest contiguous segment with monotonic condition
Main operations	Expand right, shrink left, update answer
Complexity	Usually $O(n)$
Key code idea	<code>while invalid: remove nums[left++]</code>
Edge cases	Empty array, no valid window, negative numbers

Maximum Subarray Sum

1. Overview

Maximum subarray sum asks for the largest possible sum of any contiguous subarray.

The standard algorithm is Kadane's Algorithm. It runs in $O(n)$ and is one of the most important array algorithms for interviews.

Example:

```
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
answer = 6 from [4, -1, 2, 1]
```

2. Intuition

At every index, ask:

```
Should I extend the previous subarray, or start fresh here?
```

If the previous running sum is negative, carrying it forward only hurts future sums. So we drop it and start at the current element.

Simple reasoning:

- A positive previous sum helps.
- A negative previous sum hurts.
- The best subarray ending at current index is either:
 - current element alone, or
 - previous best ending sum + current element.

Formula:

```
currentBest = max(nums[i], currentBest + nums[i])
globalBest = max(globalBest, currentBest)
```

3. When to Use It

Use Kadane's Algorithm when:

- The problem asks for maximum sum contiguous subarray.
- You need the best segment, not necessarily fixed length.
- Negative numbers may exist.
- The problem can be converted to max subarray sum.

Common trigger phrases:

- "Maximum subarray"
- "Largest sum contiguous subarray"
- "Best profit over a continuous period"
- "Maximum difference after transformation"
- "Maximum circular subarray"

4. When Not to Use It

Do not use plain Kadane when:

- The subarray length is fixed; use sliding window.
- You need non-contiguous subsequence.
- There are range updates and many queries; use Segment Tree with max prefix/suffix/subarray.
- The condition is not just sum maximization.
- You need maximum product; product needs separate min/max tracking.

Common wrong assumptions:

- Initializing answer to 0 fails when all numbers are negative.
- Returning empty subarray when problem requires non-empty subarray.
- Kadane gives the subarray itself automatically; you need extra index tracking.

5. Core Concepts

Current Best Ending Here

The maximum sum of a subarray that must end at the current index.

This matters because every global best subarray ends somewhere.

Global Best

The best value seen over all ending positions.

Restart Decision

If `currentBest + nums[i] < nums[i]`, start a new subarray at `i`.

Non-Empty Subarray

Most interview problems require at least one element. Initialize using `nums[0]`, not 0.

Index Tracking

To return boundaries, store temporary start and best start/end.

6. Step-by-Step Algorithm

1. Set `currentBest = nums[0]`.
2. Set `globalBest = nums[0]`.
3. Traverse from index 1.
4. For each element, decide whether to extend or restart: `currentBest = max(nums[i], currentBest + nums[i])`.
5. Update `globalBest = max(globalBest, currentBest)`.
6. Return `globalBest`.

7. Dry Run

Input:

```
nums = [-2, 1, -3, 4, -1, 2, 1]
```

Initial:

```
currentBest = -2
globalBest = -2
```

i	nums[i]	Extend	Start fresh	currentBest	globalBest
1	1	-1	1	1	1
2	-3	-2	-3	-2	1
3	4	2	4	4	4
4	-1	3	-1	3	4
5	2	5	2	5	5
6	1	6	1	6	6

Final answer: 6.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long maximumSubarraySum(const vector<int>& nums) {
    if (nums.empty()) return 0;

    long long currentBest = nums[0];
    long long globalBest = nums[0];

    for (int i = 1; i < (int)nums.size(); i++) {
        currentBest = max((long long)nums[i], currentBest + nums[i]);
        globalBest = max(globalBest, currentBest);
    }

    return globalBest;
}

int main() {
    int n;
    cin >> n;

    vector<int> nums(n);
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }
}
```

```
    }

    cout << maximumSubarraySum(nums) << '\n';
    return 0;
}
```

Example input:

```
9
-2 1 -3 4 -1 2 1 -5 4
```

Output:

```
6
```

pYTHON IMPLEMENTATION

```
def maximum_subarray_sum(nums):
    if not nums:
        return 0

    current_best = nums[0]
    global_best = nums[0]

    for value in nums[1:]:
        current_best = max(value, current_best + value)
        global_best = max(global_best, current_best)

    return global_best

n = int(input())
nums = list(map(int, input().split()))
print(maximum_subarray_sum(nums))
```

10. Code Explanation

- The empty case returns 0; for strict non-empty platforms, input usually has $n \geq 1$.
- `currentBest` means best subarray sum ending at current index.
- `globalBest` stores the best answer seen so far.
- For each value, `max(value, currentBest + value)` chooses whether to start fresh or extend.
- `long long` protects against overflow when many large values are added.

11. Complexity Analysis

Operation	Complexity
Preprocessing	$O(1)$
Single traversal	$O(n)$
Update per element	$O(1)$
Overall time	$O(n)$
Space	$O(1)$

12. Common Patterns

Pattern	How to identify it	General approach	Example problems
Basic max subarray	largest contiguous sum	Kadane	LeetCode 53
Circular max subarray	wrap-around allowed	max Kadane and min Kadane	LeetCode 918
Max profit	buy/sell transformed to max difference	Kadane on daily differences	LeetCode 121
Max submatrix	2D grid max sum rectangle	Fix rows + Kadane	SPOJ MMAXPER variants
Segment tree max subarray	updates + queries	store sum/prefix/suffix/best	CSES Subarray Sum Queries

13. Common Mistakes

- Initializing `globalBest = 0` for all-negative arrays.
- Forgetting that subarray must be contiguous.
- Not using `long long`.
- Incorrect circular subarray handling when all numbers are negative.
- Returning only sum when problem asks for indices.
- Resetting current sum to zero in a non-empty subarray problem without handling all-negative input.

14. Edge Cases

- Single element.
- All negative numbers.
- All positive numbers.
- Mix of zeros and negatives.
- Maximum sum at beginning.
- Maximum sum at end.
- Very large values.
- Empty input if custom function allows it.

15. Variations

Variation	What changes	Placement/CP importance
Return subarray indices	Track start/end positions	Important
Maximum circular subarray	Use total sum - minimum subarray	Important
Maximum product subarray	Track max and min product	Very important
2D maximum submatrix	Compress rows and apply Kadane	Advanced
Dynamic max subarray queries	Segment Tree node stores four values	CP important

16. Related Algorithms/Data Structures

- Prefix Sum: max subarray can be seen as maximizing `prefix[j] - minPrefixBefore`.
- Sliding Window: used for fixed length sum, not arbitrary negative arrays.
- Segment Tree: supports updates and range max subarray queries.
- Dynamic Programming: Kadane is a one-state DP.
- Monotonic Queue: used for constrained length max subarray.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Best Time to Buy and Sell Stock	LeetCode 121	Max difference/Kadane style	Easy
Maximum Subarray	GFG	Kadane basics	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Maximum Subarray	LeetCode 53	Kadane	Medium
Maximum Product Subarray	LeetCode 152	Track max/min product	Medium
Maximum Sum Circular Subarray	LeetCode 918	Kadane + min subarray	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Max Sum of Rectangle No Larger Than K	LeetCode 363	Prefix + ordered set	Hard
Subarray Sum Queries	CSES	Segment tree max subarray	Hard
Maximum Subarray Min-Product	LeetCode 1856	Monotonic stack + prefix	Hard

18. Interview Explanation

"I use Kadane's Algorithm. At each index, I maintain the best subarray sum that must end at this index. I either extend the previous subarray or start a new one from the current element. The maximum over all these ending

positions is the final answer. I initialize with the first element so all-negative arrays are handled correctly."

19. Revision Notes

- Key idea: extend or restart.
- Formula: `current = max(nums[i], current + nums[i])`.
- Answer: `best = max(best, current)`.
- Initialize from `nums[0]`.
- Complexity: $O(n)$ time, $O(1)$ space.
- Trap: all-negative arrays.

20. Final Cheat Sheet

Item	Reminder
When to use	Maximum contiguous sum
Main operations	Track current best ending here and global best
Complexity	$O(n)$ time, $O(1)$ space
Key code idea	<code>current = max(x, current + x)</code>
Edge cases	All negative, single element, overflow

Minimum Window

1. Overview

Minimum window problems ask for the smallest contiguous segment that satisfies a required condition.

The most famous example is Minimum Window Substring:

Given `s` and `t`, find the smallest substring of `s` containing all characters of `t`.

The array version may ask for:

- Minimum subarray length with sum at least `k`.
- Minimum window containing all required values.
- Minimum segment covering all distinct elements.

The common technique is sliding window with a validity check.

2. Intuition

For minimum window, we usually:

1. Expand right until the window becomes valid.
2. Shrink left as much as possible while keeping it valid.

3. Record the smallest valid window.

Analogy: stretch a rubber band until it covers everything required, then tighten from the left to make it as small as possible.

Why it works:

- Expanding right helps include missing requirements.
- Shrinking left removes unnecessary elements.
- Every pointer moves forward only, giving linear time for many conditions.

3. When to Use It

Use minimum window when:

- You need the shortest contiguous segment.
- A window can be checked as valid/invalid.
- Adding/removing elements can update validity efficiently.
- The condition is based on counts, coverage, or positive sum.

Common trigger phrases:

- "Minimum window"
- "Smallest subarray"
- "Shortest substring"
- "Contains all characters"
- "At least K"
- "Cover all elements"
- "Minimum length subarray"

4. When Not to Use It

Avoid simple sliding window when:

- Negative values exist in sum-at-least problems.
- The condition cannot be updated locally.
- You need non-contiguous selection.
- The required ordering is complex and not window-based.

Alternatives:

- Prefix sum + monotonic deque for shortest subarray sum at least k with negatives.
- Binary search + feasibility check for some minimum length problems.
- Hash map and sorting for non-contiguous matching.

5. Core Concepts

Need Counts

For character/value coverage, store required frequency.

Example:

```
t = "AABC"
need[A] = 2, need[B] = 1, need[C] = 1
```

Window Counts

Track frequencies inside current window.

Formed Requirement

Count how many required keys currently satisfy their needed frequency.

This avoids checking the full map repeatedly.

Shrink While Valid

For minimum windows, the answer is updated inside the shrink loop because every valid window might be the smallest.

Positive Sum Minimum Window

For `sum >= k` with positive numbers, use running sum and shrink while `sum >= k`.

6. Step-by-Step Algorithm

For minimum window substring:

1. Count required characters in `target`.
2. Set `required = number of unique required characters`.
3. Use two pointers `left = 0, right = 0`.
4. Expand `right`, adding characters to window count.
5. If a character count reaches its required frequency, increment `formed`.
6. While `formed == required`, update best answer and remove `s[left]`.
7. If removing breaks a requirement, decrement `formed`.
8. Continue until `right` reaches the end.
9. Return the best window.

7. Dry Run

Input:

```
s = "ADOBECODEBANC"
t = "ABC"
```

Need:

A:1, B:1, C:1

Important steps:

Step	Window	Action	formed	Best
Expand to C	ADOBEC	all A,B,C found	3	ADOBEC
Shrink left	remove A	invalid	2	ADOBEC
Expand to A	DOBECODEBA	A found again	3	ADOBEC
Shrink	remove extra chars	still valid	3	improves
Reach C	BANC	valid and smaller	3	BANC

Final answer: "BANC".

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

string minWindow(string s, string t) {
    if (s.empty() || t.empty()) return "";

    unordered_map<char, int> need;
    for (char c : t) {
        need[c]++;
    }

    unordered_map<char, int> window;
    int required = (int)need.size();
    int formed = 0;

    int left = 0;
    int bestLength = INT_MAX;
    int bestStart = 0;

    for (int right = 0; right < (int)s.size(); right++) {
        char added = s[right];
        window[added]++;

        if (need.count(added) && window[added] == need[added]) {
            formed++;
        }

        while (left <= right && formed == required) {
            int currentLength = right - left + 1;
            if (currentLength < bestLength) {
                bestLength = currentLength;
            }
        }
    }
}
```

```

        bestStart = left;
    }

    char removed = s[left];
    window[removed]--;
    if (need.count(removed) && window[removed] < need[removed]) {
        formed--;
    }
    left++;
}

if (bestLength == INT_MAX) return "";
return s.substr(bestStart, bestLength);
}

int main() {
    string s, t;
    cin >> s >> t;

    cout << minWindow(s, t) << '\n';
    return 0;
}

```

Example input:

```
ADOBECODEBANC ABC
```

Output:

```
BANC
```

pYTHON IMPLEMENTATION

```

from collections import Counter, defaultdict

def min_window(s, t):
    if not s or not t:
        return ""

    need = Counter(t)
    window = defaultdict(int)
    required = len(need)
    formed = 0

    left = 0

```

```

best_length = float("inf")
best_start = 0

for right, char in enumerate(s):
    window[char] += 1

    if char in need and window[char] == need[char]:
        formed += 1

    while left <= right and formed == required:
        current_length = right - left + 1
        if current_length < best_length:
            best_length = current_length
            best_start = left

        removed = s[left]
        window[removed] -= 1
        if removed in need and window[removed] < need[removed]:
            formed -= 1
        left += 1

    if best_length == float("inf"):
        return ""
    return s[best_start:best_start + best_length]

s, t = input().split()
print(min_window(s, t))

```

10. Code Explanation

- `need` stores required frequencies from `t`.
- `window` stores frequencies in the current substring.
- `required` is the number of distinct required characters.
- `formed` counts how many distinct required characters currently have enough frequency.
- The outer loop expands the window.
- The inner `while` loop shrinks the window while it remains valid.
- `bestLength` and `bestStart` store the smallest valid answer.

11. Complexity Analysis

Operation	Complexity
Build need map	$O(m)$
Expand right pointer	$O(n)$ total
Shrink left pointer	$O(n)$ total
Overall time	$O(n + m)$
Space	$O(\text{unique characters})$

12. Common Patterns

Pattern	How to identify it	General approach	Example problems
Minimum covering substring	contains all chars of another string	Sliding window + counts	LeetCode 76
Minimum size sum	sum at least target, positives	Sliding window sum	LeetCode 209
Cover all distinct values	smallest segment containing all types	Window + frequency map	GFG smallest distinct window
Shortest subarray sum at least K with negatives	negatives allowed	Prefix + monotonic deque	LeetCode 862

13. Common Mistakes

- Updating answer only after the shrink loop.
- Counting total matched characters incorrectly when duplicates exist.
- Forgetting that `t = "AABC"` needs two `A`s.
- Removing from left without updating validity.
- Using sliding window for sum with negative numbers.
- Returning wrong substring length due to off-by-one error.

14. Edge Cases

- Empty source string.
- Empty target string.
- Target longer than source.
- No valid window.
- Duplicate required characters.
- Answer at the beginning.
- Answer at the end.
- Case-sensitive characters.

15. Variations

Variation	What changes	Placement/CP importance
Minimum size subarray sum	Track sum instead of char counts	Very important
Minimum window subsequence	Order matters but not contiguous in target	Advanced
Shortest subarray sum at least K with negatives	Use monotonic deque	Very important
Smallest range covering K lists	Use heap/sliding sorted values	Advanced

16. Related Algorithms/Data Structures

- Sliding Window: core technique.
- Hash Map: frequency tracking.
- Monotonic Deque: needed for negative-sum shortest subarray variants.
- Two Pointers: general framework.
- Heap: useful for smallest range covering multiple sorted lists.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Minimum Size Subarray Sum with positives variant	GFG	Sliding sum window	Easy
Smallest Window containing 0, 1 and 2	GFG	Frequency window	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Minimum Size Subarray Sum	LeetCode 209	Positive sum window	Medium
Find All Anagrams in a String	LeetCode 438	Fixed window counts	Medium
Permutation in String	LeetCode 567	Window frequency match	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Minimum Window Substring	LeetCode 76	Dynamic valid window	Hard
Shortest Subarray with Sum at Least K	LeetCode 862	Prefix + monotonic deque	Hard
Smallest Range Covering Elements from K Lists	LeetCode 632	Heap/window coverage	Hard

18. Interview Explanation

"I use a sliding window. I expand the right pointer until the window satisfies all requirements, then I shrink from the left while the window remains valid. During shrinking, I update the best answer because that is where the minimum window is found. I maintain counts and a matched/formed variable so validity checks are constant time."

19. Revision Notes

- Key idea: expand until valid, shrink while valid.
- For duplicates, compare frequencies, not just presence.
- Update answer inside the valid shrink loop.
- Use `formed == required` for efficient validity.
- Complexity: $O(n + m)$.

- Trap: negative numbers in sum windows require different technique.

20. Final Cheat Sheet

Item	Reminder
When to use	Smallest contiguous segment satisfying coverage/positive-sum condition
Main operations	Expand right, shrink left while valid
Complexity	Usually $O(n)$
Key code idea	<code>while valid: update answer; remove s[left++]</code>
Edge cases	Duplicates, no answer, target longer than source

Merge Intervals

1. Overview

Merge intervals is a sorting-based pattern used to combine overlapping ranges.

An interval is usually represented as:

```
[start, end]
```

Two intervals overlap if the next start is less than or equal to the current end:

```
nextStart <= currentEnd
```

This pattern is common in scheduling, calendar, meeting rooms, range compression, and timeline problems.

2. Intuition

If intervals are unsorted, overlap relationships are hard to see.

After sorting by start time, intervals that can overlap will appear next to each other.

Then we scan from left to right:

- If the next interval overlaps with the current merged interval, extend the end.
- Otherwise, push the current interval and start a new one.

Analogy: place all time slots on a timeline from left to right. Whenever two slots touch or overlap, paint them as one continuous block.

3. When to Use It

Use merge intervals when:

- You have ranges with start and end.
- You need to combine overlaps.
- You need to count non-overlapping groups.
- You need to insert a new interval.
- You need to find gaps between intervals.

Common trigger phrases:

- "Merge overlapping intervals"
- "Meeting schedule"
- "Calendar booking"
- "Insert interval"
- "Non-overlapping intervals"
- "Minimum arrows"
- "Employee free time"

4. When Not to Use It

Do not use basic merge intervals when:

- You need frequent dynamic insert/delete operations; use balanced BST or interval tree.
- Intervals are points only; sorting or counting may be enough.
- You need maximum overlap count; use sweep line.
- You need range sum updates; use difference array or segment tree.
- The intervals are already guaranteed non-overlapping and sorted; merging may be unnecessary.

5. Core Concepts

Sorting by Start

Sort intervals by **start**, then by **end** if needed.

This matters because potential overlaps become adjacent.

Overlap Condition

For closed intervals:

```
next.start <= current.end
```

For half-open intervals [**start**, **end**), touching intervals may not overlap:

```
next.start < current.end
```

Merge Operation

When overlapping:

```
current.end = max(current.end, next.end)
```

Gap

If `next.start > current.end`, there is no overlap.

Interval Boundary Type

Always check whether intervals are closed or half-open. Meeting room problems often treat `[end, nextStart]` as non-overlapping if `end <= nextStart`.

6. Step-by-Step Algorithm

1. If interval list is empty, return empty result.
2. Sort intervals by start.
3. Set `current` to the first interval.
4. Traverse remaining intervals.
5. If the next interval overlaps with `current`, update `current.end`.
6. Otherwise, push `current` to result and set `current = next`.
7. After the loop, push the last `current`.
8. Return result.

7. Dry Run

Input:

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
```

After sorting: same order.

Step	Current merged	Next interval	Overlap?	Action	Result
1	[1,3]	[2,6]	yes	extend to [1,6]	[]
2	[1,6]	[8,10]	no	push [1,6]	[[1,6]]
3	[8,10]	[15,18]	no	push [8,10]	[[1,6], [8,10]]
End	[15,18]	none	-	push last	[[1,6], [8,10], [15,18]]

Final answer:

```
[[1,6], [8,10], [15,18]]
```

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> mergeIntervals(vector<vector<int>> intervals) {
    if (intervals.empty()) return {};

    sort(intervals.begin(), intervals.end());

    vector<vector<int>> merged;
    vector<int> current = intervals[0];

    for (int i = 1; i < (int)intervals.size(); i++) {
        int nextStart = intervals[i][0];
        int nextEnd = intervals[i][1];

        if (nextStart <= current[1]) {
            current[1] = max(current[1], nextEnd);
        } else {
            merged.push_back(current);
            current = intervals[i];
        }
    }

    merged.push_back(current);
    return merged;
}

int main() {
    int n;
    cin >> n;

    vector<vector<int>> intervals(n, vector<int>(2));
    for (int i = 0; i < n; i++) {
        cin >> intervals[i][0] >> intervals[i][1];
    }

    vector<vector<int>> answer = mergeIntervals(intervals);
    for (const auto& interval : answer) {
        cout << interval[0] << ' ' << interval[1] << '\n';
    }

    return 0;
}
```

Example input:

```
4
1 3
```

```
2 6
8 10
15 18
```

Output:

```
1 6
8 10
15 18
```

pYTHON IMPLEMENTATION

```
def merge_intervals(intervals):
    if not intervals:
        return []

    intervals.sort()
    merged = []
    current_start, current_end = intervals[0]

    for next_start, next_end in intervals[1:]:
        if next_start <= current_end:
            current_end = max(current_end, next_end)
        else:
            merged.append([current_start, current_end])
            current_start, current_end = next_start, next_end

    merged.append([current_start, current_end])
    return merged

n = int(input())
intervals = [list(map(int, input().split())) for _ in range(n)]
for start, end in merge_intervals(intervals):
    print(start, end)
```

10. Code Explanation

- The function takes intervals by value in C++ so sorting does not mutate the caller's vector.
- `sort(intervals.begin(), intervals.end())` sorts by start, then end.
- `current` stores the interval currently being built.
- If `nextStart <= current[1]`, intervals overlap and we extend the end.
- If not, the current merged interval is complete and pushed to `merged`.
- The last current interval is pushed after the loop.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n)$
Merge scan	$O(n)$
Overall time	$O(n \log n)$
Extra space excluding output	$O(1)$ or $O(\log n)$ depending on sort
Output space	$O(n)$

12. Common Patterns

Pattern	How to identify it	General approach	Example problems
Merge overlaps	"merge intervals"	Sort by start + scan	LeetCode 56
Insert interval	sorted intervals + one new range	Merge around inserted range	LeetCode 57
Meeting rooms	detect overlap	Sort by start/end	LeetCode 252, 253
Remove overlaps	maximize non-overlap	Sort by end greedily	LeetCode 435
Burst balloons	intervals overlap by arrow point	Sort by end	LeetCode 452

13. Common Mistakes

- Forgetting to sort first.
- Using wrong overlap condition for closed vs half-open intervals.
- Not pushing the last interval.
- Sorting by end when merge requires sorting by start.
- Mutating intervals unexpectedly when caller expects original order.
- Overflow when interval endpoints are huge and midpoint is computed elsewhere.

14. Edge Cases

- Empty interval list.
- Single interval.
- All intervals overlap.
- No intervals overlap.
- Touching intervals like $[1,4]$ and $[4,5]$.
- Duplicate intervals.
- Negative endpoints.
- Intervals given in random order.

15. Variations

Variation	What changes	Placement/CP importance
Insert interval	Add one interval then merge affected range	Very important
Meeting Rooms II	Need max simultaneous overlaps	Very important
Non-overlapping intervals	Sort by end and greedily remove	Important
Employee free time	Merge busy intervals then find gaps	Important
Interval intersection	Two sorted lists with two pointers	Important

16. Related Algorithms/Data Structures

- Sorting: required to order intervals.
- Sweep Line: better for maximum overlap and event counting.
- Heap: used in meeting room allocation.
- Balanced BST: dynamic interval insertion/search.
- Difference Array: useful for dense coordinate range updates.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Meeting Rooms	LeetCode 252	Sort and detect overlap	Easy
Check if Intervals Overlap	GFG	Basic overlap condition	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Merge Intervals	LeetCode 56	Sort + merge scan	Medium
Insert Interval	LeetCode 57	Merge around inserted interval	Medium
Non-overlapping Intervals	LeetCode 435	Greedy by end time	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Employee Free Time	LeetCode 759	Merge schedules and find gaps	Hard
Meeting Rooms III	LeetCode 2402	Heap simulation	Hard
Count Integers in Intervals	LeetCode 2276	Ordered intervals dynamically	Hard

18. Interview Explanation

"I sort intervals by start time because after sorting, overlapping intervals must appear adjacent. Then I keep one current merged interval. If the next interval starts before or at the current end, I merge by extending the

end. Otherwise, I save the current interval and start a new one. The scan is linear after sorting."

19. Revision Notes

- Key idea: sort by start, then scan.
- Overlap for closed intervals: $\text{nextStart} \leq \text{currentEnd}$.
- Merge end: $\text{currentEnd} = \max(\text{currentEnd}, \text{nextEnd})$.
- Push the last interval.
- Complexity: $O(n \log n)$.
- Trap: interval boundary convention.

20. Final Cheat Sheet

Item	Reminder
When to use	Ranges, schedules, overlapping segments
Main operations	Sort, compare, merge
Complexity	$O(n \log n)$ time
Key code idea	If overlap, extend end; else push current
Edge cases	Empty list, touching ranges, duplicates

Sort By Custom Condition

1. Overview

Sort by custom condition means ordering elements using a comparator instead of default ascending order.

This appears often in placement and CP problems where sorting is only the first step before a greedy, interval, string, or ranking solution.

Examples:

- Sort intervals by end time.
- Sort people by height descending and index ascending.
- Sort numbers to form the largest number.
- Sort pairs by first ascending and second descending.
- Sort jobs by profit, deadline, or ratio.

2. Intuition

Default sorting answers:

Which value is smaller?

Custom sorting answers:

Which element should come first for my strategy?

For greedy problems, the comparator encodes the greedy priority.

Example:

For activity selection, sorting by earliest ending time works because picking the interval that ends earliest leaves maximum room for future intervals.

Important rule:

Your comparator must define a consistent strict ordering. In C++, `comp(a, b)` should return `true` only when `a` must come before `b`.

3. When to Use It

Use custom sorting when:

- Objects have multiple fields.
- Tie-breaking matters.
- Greedy choice depends on a specific priority.
- Default ascending order is not enough.
- You need lexicographic-like control.

Common trigger phrases:

- "Sort by frequency"
- "If tie, sort by..."
- "Arrange to form largest number"
- "Minimum number of arrows"
- "Maximum chain length"
- "Sort intervals by ending time"
- "Custom comparator"

4. When Not to Use It

Avoid custom sorting when:

- A simple default sort is enough.
- The data must support frequent online insertions; use heap or balanced BST.
- You need stable ordering but use unstable sort without handling ties.
- Comparator is non-transitive.
- You are sorting just to find top K; heap or selection may be better.

Common wrong assumptions:

- Returning `a <= b` in C++ comparator is okay. It is not.
- Tie-breakers are optional. They may be necessary for correctness.
- Sorting by the largest immediate value always proves a greedy solution.

5. Core Concepts

Comparator

A function deciding whether **a** should come before **b**.

In C++:

```
sort(v.begin(), v.end(), [](auto& a, auto& b) {  
    return a.first < b.first;  
});
```

Strict Weak Ordering

Comparator must be consistent:

- `comp(a, a)` should be false.
- If **a** comes before **b**, **b** should not come before **a**.
- Ordering should be transitive.

Tie-Breaking

When primary fields are equal, use secondary fields.

Example:

```
sort by height descending, k ascending
```

Stable Sort

`stable_sort` preserves original relative order among equivalent elements.

Use it only when original order matters.

Greedy Proof

Custom sorting in greedy problems should have a reason, usually exchange argument or leaving maximum future options.

6. Step-by-Step Algorithm

For sorting intervals by end time:

1. Store intervals as pairs or vectors.
2. Define comparator:
 - Smaller end comes first.
 - If ends are equal, smaller start comes first.
3. Sort using comparator.

4. Use sorted order in the greedy algorithm.
5. Process elements in sorted order.

7. Dry Run

Input intervals:

```
[[1,3], [2,2], [3,4], [1,2]]
```

Sort by end ascending, then start ascending.

Interval	End	Start	Sorted position
[1,2]	2	1	1
[2,2]	2	2	2
[1,3]	3	1	3
[3,4]	4	3	4

Final sorted order:

```
[[1,2], [2,2], [1,3], [3,4]]
```

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Interval {
    int start;
    int end;
};

vector<Interval> sortIntervalsByEnd(vector<Interval> intervals) {
    sort(intervals.begin(), intervals.end(), [](const Interval& a, const Interval&
b) {
        if (a.end != b.end) {
            return a.end < b.end;
        }
        return a.start < b.start;
    });

    return intervals;
}

int main() {
```

```

int n;
cin >> n;

vector<Interval> intervals(n);
for (int i = 0; i < n; i++) {
    cin >> intervals[i].start >> intervals[i].end;
}

vector<Interval> sortedIntervals = sortIntervalsByEnd(intervals);
for (const Interval& interval : sortedIntervals) {
    cout << interval.start << ' ' << interval.end << '\n';
}

return 0;
}

```

Example input:

```

4
1 3
2 2
3 4
1 2

```

Output:

```

1 2
2 2
1 3
3 4

```

pYTHON IMPLEMENTATION

```

def sort_intervals_by_end(intervals):
    return sorted(intervals, key=lambda interval: (interval[1], interval[0]))

n = int(input())
intervals = [tuple(map(int, input().split())) for _ in range(n)]

for start, end in sort_intervals_by_end(intervals):
    print(start, end)

```

10. Code Explanation

- **Interval** stores named fields, making the comparator readable.
- The comparator first checks **end**.
- If endings differ, the smaller ending interval comes first.
- If endings are equal, smaller **start** comes first.
- The function takes input by value to avoid changing the original vector.
- Python uses a tuple key (**end**, **start**) to express the same ordering.

11. Complexity Analysis

Operation	Complexity
Comparator call	$O(1)$ for simple fields
Sorting	$O(n \log n)$
Extra space in C++ sort	$O(\log n)$ typical recursion/introsort
Python sorted space	$O(n)$
Overall time	$O(n \log n)$

12. Common Patterns

Pattern	How to identify it	General approach	Example problems
Sort by end	interval scheduling	Greedy earliest finish	Activity Selection, LeetCode 435
Sort by start	merge or scan intervals	Sort then scan	LeetCode 56
Sort by two keys	"if tie..."	Primary + secondary key	Queue Reconstruction
Largest number	concatenate comparison	Compare a+b vs b+a	LeetCode 179
Frequency sort	sort by count then value	Map + comparator	LeetCode 451

13. Common Mistakes

- Using **<=** in C++ comparator.
- Forgetting tie-breakers.
- Creating a comparator that is not transitive.
- Capturing references to temporary values.
- Sorting numbers as strings without correct concatenation logic.
- Assuming **sort** is stable.
- Overflow in comparators like **return a - b**.

14. Edge Cases

- Empty list.
- Single element.

- All keys equal.
- Duplicate objects.
- Negative values.
- Very large values.
- Strings with different lengths.
- Comparator tie cases.

15. Variations

Variation	What changes	Placement/CP importance
Multi-key sort	Use tuple/comparator tie-breaks	Very important
Descending sort	Reverse comparison	Very important
Stable sort	Preserve original equivalent order	Situational
Custom object sort	Comparator accesses fields	Important
Concatenation sort	Compare combined strings	Important

16. Related Algorithms/Data Structures

- Greedy Algorithms: custom sort often defines greedy order.
- Heap: better for online top-K or repeated min/max extraction.
- Balanced BST: keeps data sorted dynamically.
- Sweep Line: sorts events by coordinate and event type.
- Priority Queue: custom comparator for dynamic ordering.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Sort Array by Increasing Frequency	LeetCode 1636	Frequency + value tie-break	Easy
Relative Sort Array	LeetCode 1122	Custom order map	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Largest Number	LeetCode 179	String concatenation comparator	Medium
Queue Reconstruction by Height	LeetCode 406	Sort by height desc, k asc	Medium
Non-overlapping Intervals	LeetCode 435	Sort by end for greedy	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Russian Doll Envelopes	LeetCode 354	Sort width asc, height desc + LIS	Hard
Maximum Performance of a Team	LeetCode 1383	Sort by efficiency + heap	Hard
The Skyline Problem	LeetCode 218	Event sorting + heap/multiset	Hard

18. Interview Explanation

"When default order is not enough, I define a comparator based on the property my algorithm needs. I make sure the comparator is strict and handles ties correctly. For greedy problems, the sort order must be justified, such as sorting intervals by end time because finishing earlier leaves more room for future intervals."

19. Revision Notes

- Key idea: comparator encodes priority.
- C++ comparator must use `<`, not `<=`.
- Always handle ties deliberately.
- Complexity: $O(n \log n)$.
- For Python, use `key=`.
- Trap: non-transitive comparator causes undefined or wrong behavior.

20. Final Cheat Sheet

Item	Reminder
When to use	Multi-field objects, tie rules, greedy order
Main operations	Define comparator/key, sort, process
Complexity	$O(n \log n)$
Key code idea	<code>if primary differs return primary rule; else tie rule</code>
Edge cases	Equal keys, duplicates, stability, overflow

Prefix/Suffix Contribution

1. Overview

Prefix/suffix contribution is a pattern where each element's contribution to the final answer is calculated using information before it and after it.

Instead of recomputing left and right parts repeatedly, we precompute:

- Prefix values: information from the left.
- Suffix values: information from the right.

Common examples:

- Product of array except self.

- Trapping rain water.
- Sum of absolute differences in sorted array.
- Contribution of each element as minimum/maximum.
- Count/sum of subarrays where an element contributes.

2. Intuition

Many problems ask:

For each index i , what is the effect of elements before i and after i ?

Naively, you might loop left and right for every i , causing $O(n^2)$.

Prefix/suffix arrays save repeated work:

- Prefix remembers answers from the left.
- Suffix remembers answers from the right.
- Combine them at each index.

Analogy: instead of asking every person in a line to count everyone before and after them from scratch, we give each person two labels: count before and count after.

3. When to Use It

Use this pattern when:

- Each index needs information from left and right.
- Repeated range computation appears.
- The operation can be accumulated.
- You need all answers, one per index.
- You need to count contribution of each element to a global answer.

Common trigger phrases:

- "Except self"
- "For every index"
- "Contribution of each element"
- "Sum over all subarrays"
- "Left and right"
- "Prefix and suffix"
- "Trapping rain water"

4. When Not to Use It

Avoid prefix/suffix arrays when:

- You only need one simple range query; direct computation may be enough.
- Updates happen frequently; use Fenwick Tree or Segment Tree.
- The operation is not associative or cannot be accumulated.

- A monotonic stack is actually needed for nearest smaller/greater contribution.
- Memory is tight and one-pass optimization is possible.

Common wrong assumptions:

- Prefix/suffix always means sum; it can be max, min, product, gcd, count, etc.
- Division is safe for product except self. It fails with zeros.
- Contribution formulas do not need boundary care. They usually do.

5. Core Concepts

Prefix Array

`prefix[i]` stores information from `0..i` or sometimes `0..i-1`.

For product except self, exclusive prefix is useful:

```
prefixProductBefore[i] = product of nums[0..i-1]
```

Suffix Array

`suffix[i]` stores information from `i..n-1` or sometimes `i+1..n-1`.

Contribution

Contribution means how much one element adds to the final answer.

Example:

In sorted array absolute differences:

```
left contribution = nums[i] * i - sumLeft
right contribution = sumRight - nums[i] * (n - i - 1)
```

Inclusive vs Exclusive

Always define whether prefix includes current element.

Space Optimization

Sometimes suffix can be tracked with one variable while scanning from right.

6. Step-by-Step Algorithm

For Product of Array Except Self:

1. Create `answer` array of size `n`, initialized to `1`.
2. Scan left to right with `prefixProduct`.

3. Set `answer[i] = prefixProduct`.
4. Multiply `prefixProduct *= nums[i]`.
5. Scan right to left with `suffixProduct`.
6. Multiply `answer[i] *= suffixProduct`.
7. Multiply `suffixProduct *= nums[i]`.
8. Return `answer`.

7. Dry Run

Input:

```
nums = [1, 2, 3, 4]
```

Left pass:

i	nums[i]	prefixProduct before	answer[i]	prefixProduct after
0	1	1	1	1
1	2	1	1	2
2	3	2	2	6
3	4	6	6	24

Right pass:

i	nums[i]	suffixProduct before	answer[i] after multiply	suffixProduct after
3	4	1	6	4
2	3	4	8	12
1	2	12	12	24
0	1	24	24	24

Final answer:

```
[24, 12, 8, 6]
```

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> productExceptSelf(const vector<int>& nums) {
    int n = (int)nums.size();
```

```
vector<long long> answer(n, 1);

long long prefixProduct = 1;
for (int i = 0; i < n; i++) {
    answer[i] = prefixProduct;
    prefixProduct *= nums[i];
}

long long suffixProduct = 1;
for (int i = n - 1; i >= 0; i--) {
    answer[i] *= suffixProduct;
    suffixProduct *= nums[i];
}

return answer;
}

int main() {
    int n;
    cin >> n;

    vector<int> nums(n);
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    vector<long long> answer = productExceptSelf(nums);
    for (long long value : answer) {
        cout << value << ' ';
    }
    cout << '\n';

    return 0;
}
```

Example input:

```
4
1 2 3 4
```

Output:

```
24 12 8 6
```

pYTHON IMPLEMENTATION

```
def product_except_self(nums):
    n = len(nums)
    answer = [1] * n

    prefix_product = 1
    for i in range(n):
        answer[i] = prefix_product
        prefix_product *= nums[i]

    suffix_product = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix_product
        suffix_product *= nums[i]

    return answer

n = int(input())
nums = list(map(int, input().split()))
print(*product_except_self(nums))
```

10. Code Explanation

- `answer[i]` first stores the product of all elements before `i`.
- `prefixProduct` is updated after assignment, so it excludes the current element.
- The right-to-left pass multiplies by product of all elements after `i`.
- `suffixProduct` is updated after multiplication, so it also excludes current element.
- This avoids division and handles zeros correctly.

11. Complexity Analysis

Operation	Complexity
Left prefix pass	$O(n)$
Right suffix pass	$O(n)$
Overall time	$O(n)$
Extra space excluding output	$O(1)$
Output space	$O(n)$

12. Common Patterns

Pattern	How to identify it	General approach	Example problems
Except self	result for each index excluding itself	Prefix + suffix	LeetCode 238

Pattern	How to identify it	General approach	Example problems
Left/right max	water trapped depends on both sides	Prefix max + suffix max	LeetCode 42
Sorted absolute difference	contribution from left and right sums	Prefix sums	LeetCode 1685
Sum over subarrays	each element contributes many times	Count choices left/right	Sum of Subarray Minimums
Equilibrium index	left sum equals right sum	Prefix total relation	GFG equilibrium point

13. Common Mistakes

- Confusing inclusive and exclusive prefix.
- Using division in product except self and failing with zeros.
- Forgetting to use `long long`.
- Building both prefix and suffix arrays when one variable is enough.
- Wrong contribution count by one.
- Not handling empty or single-element arrays.

14. Edge Cases

- Empty array.
- Single element.
- Contains one zero.
- Contains multiple zeros.
- Negative values.
- Large products/sums.
- All equal values.
- Sorted and reverse sorted input.

15. Variations

Variation	What changes	Placement/CP importance
Prefix/suffix max	Store max to left/right	Very important
Prefix/suffix gcd	Range gcd except self	CP important
Contribution with monotonic stack	Find span where element is min/max	Very important
Difference array contribution	Range add effects	Important
Prefix count contribution	Count smaller/greater before index	CP important

16. Related Algorithms/Data Structures

- Prefix Sum: most common contribution helper.
- Suffix Array in CP string context: unrelated name, do not confuse.
- Monotonic Stack: finds contribution boundaries for min/max subarray problems.

- Fenwick Tree: dynamic prefix contribution counts.
- Segment Tree: dynamic range contribution queries.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Find Pivot Index	LeetCode 724	Left sum vs right sum	Easy
Running Sum of 1d Array	LeetCode 1480	Prefix basics	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Product of Array Except Self	LeetCode 238	Exclusive prefix/suffix product	Medium
Trapping Rain Water	LeetCode 42	Left max and right max	Medium/Hard
Sum of Absolute Differences in a Sorted Array	LeetCode 1685	Prefix contribution formula	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Sum of Subarray Minimums	LeetCode 907	Monotonic stack contribution	Hard
Maximum Subarray Min-Product	LeetCode 1856	Prefix sum + stack contribution	Hard
Count of Smaller Numbers After Self	LeetCode 315	Fenwick/merge contribution counts	Hard

18. Interview Explanation

"I use prefix/suffix contribution when each index needs information from both sides. I precompute or carry prefix information from the left and suffix information from the right, then combine them for each index. This avoids recomputing ranges repeatedly and usually reduces $O(n^2)$ logic to $O(n)$."

19. Revision Notes

- Key idea: left contribution + right contribution.
- Decide inclusive vs exclusive prefix clearly.
- For product except self, avoid division.
- Use `long long` for large sums/products.
- Complexity: often $O(n)$.
- Trap: off-by-one in contribution count.

20. Final Cheat Sheet

Item	Reminder
When to use	Each index depends on left and right information
Main operations	Build/carry prefix and suffix values
Complexity	Usually $O(n)$
Key code idea	First pass stores left, second pass combines right
Edge cases	Zeros, single element, overflow, inclusive/exclusive boundaries